

CHOFYAI STUDIO

10 · Configuración

Documento	10-configuration.md
Versión analizada	0.5.1 (commit f840055)
Fecha del análisis	2026-08-28
Fuente Markdown	docs/system-documentation/10-configuration.md
Generado por	scripts/docs/build-pdf.mjs

Contenido

1. Mapa de archivos de configuración
2. ``storage/state/settings.json``
3. Variables de entorno
4. Claves de ``localStorage``
5. Configuración del frontend
 - 5.1 ``package.json``
 - 5.2 ``npmrc``
 - 5.3 ``tsconfig.json``
 - 5.4 ``vite.config.ts``
6. Configuración del backend y del empaquetado
 - 6.1 ``src-tauri/Cargo.toml``
 - 6.2 ``src-tauri/tauri.conf.json``
 - 6.3 ``src-tauri/tauri.macos.conf.json``
 - 6.4 ``src-tauri/capabilities/default.json``
 - 6.5 ``Info.plist`` y ``Entitlements.plist``
 - 6.6 ``cargo/config.toml``
7. Configuración de calidad y automatización
8. Diferencias entre entornos
9. Banderas de comportamiento
10. Configuraciones sensibles y consecuencias de un error
11. Gestión de secretos

Estado: completo · Última revisión: 2026-08-27 · Versión analizada: 0.5.1 (commit f840055)

Inventario de todo lo configurable del sistema: qué archivo, qué campo, quién lo lee, qué pasa si falta y qué se rompe si está mal. La instalación paso a paso está en [02 · Instalación y ejecución](#).

1. Mapa de archivos de configuración

Archivo	Ámbito	Lo consume
package.json	Frontend y tareas	pnpm, Vite, Vitest, CI
pnpm-lock.yaml	Dependencias exactas	pnpm con <code>--frozen-lockfile</code>
.npmrc	Registro y endurecimiento de pnpm	pnpm
tsconfig.json	Compilación de TypeScript	<code>tsc --noEmit</code> , Vite
vite.config.ts	Servidor de desarrollo y build	Vite
src-tauri/Cargo.toml	Paquete y dependencias Rust	Cargo
src-tauri/tauri.conf.json	Configuración base de Tauri	Tauri CLI y runtime
src-tauri/tauri.macos.conf.json	Sobrescritura para el empaquetado de macOS	<code>pnpm tauri:build:mac</code>
src-tauri/capabilities/default.json	Permisos del WebView	Runtime de Tauri
src-tauri/Info.plist	Metadatos del <code>.app</code>	macOS
src-tauri/Entitlements.plist	Permisos de firma	Firma de código de macOS
.cargo/config.toml	Directorio de build de Cargo	Cargo
.markdownlint.json, .markdownlint-cli2.jsonc, .markdownlintignore	Reglas de Markdown	markdownlint-cli2, CI
.gitignore	Exclusiones del repositorio	Git
.github/dependabot.yml	Actualización de dependencias	Dependabot
storage/state/settings.json	Estado del usuario	Backend Rust y scripts
apps/*.yaml	Catálogo de herramientas	<code>collect_manifests</code>
marketplace/registry.yaml	Catálogo importable	<code>list_marketplace_tools</code>
workflows/*.yaml	Pipelines	<code>list_workflows</code>

2. storage/state/settings.json

Es el único archivo de configuración que el usuario modifica de forma habitual, casi siempre sin editarlo a mano. Su esquema es exactamente la struct `AppSettings` de `src-tauri/src/models.rs`.

Campo	Tipo	Obligatorio	Valor por defecto	Efecto
<code>studio_home</code>	<code>String</code>	sí	<code>\$HOME/ChofyAISTudio</code>	Raíz donde se instalan y ejecutan las herramientas
<code>tool_overrides</code>	<code>Map<String, String></code>	no	<code>{}</code>	Ruta alternativa por herramienta; absoluta o relativa al home
<code>fallback_home</code>	<code>`String \`</code>	<code>null</code>	no	<code>null</code>
<code>sparsebundle_path</code>	<code>`String \`</code>	<code>null</code>	no	<code>null</code>
<code>models_dir</code>	<code>`String \`</code>	<code>null</code>	no	<code>null</code>
<code>outputs_dir</code>	<code>`String \`</code>	<code>null</code>	no	<code>null</code>
<code>cache_dir</code>	<code>`String \`</code>	<code>null</code>	no	<code>null</code>

Ejemplo con valores ficticios:

```
{
  "studio_home": "/Volumes/EJEMPLO/ChofyAIStudio",
  "tool_overrides": {
    "comfyui": "/Volumes/OtroDisco/modules/comfyui"
  },
  "fallback_home": null,
  "sparsebundle_path": "/Volumes/EJEMPLO/ChofyAIStudio.sparsebundle",
  "models_dir": null,
  "outputs_dir": null,
  "cache_dir": null
}
```

Comportamiento verificado:

- Todos los campos salvo `studio_home` llevan `#[serde(default)]`, así que un archivo antiguo sin ellos se lee sin problema.
- Si el JSON está corrupto o falta `studio_home`, `load_settings` **no falla**: devuelve la configuración por defecto completa. El usuario pierde su configuración sin recibir aviso.
- Quién escribe este archivo: `save_studio_home`, `save_path_settings`, `relocate_module` y `clear_module_override`, todos a través de `save_settings_to_disk`, que serializa con formato legible y hace un `fs::write` **no atómico**.
- Dónde vive: ejecutando desde el repositorio, en `storage/state/settings.json`; desde el `.app` empaquetado, en `<app_data_dir>/state/settings.json`. Lo decide `app_paths()`.

Junto a este archivo se escribe `processes.json` (mapa `tool_id` → PID) y `crash.log`.

3. Variables de entorno

Variable	Quién la define	Quién la lee	Efecto
CHOFYAI_STUDIO_HOME	El backend, antes de lanzar cualquier script	<code>resolve_studio_home</code> en <code>scripts/mac/common.sh</code> y <code>ResolveStudioHome</code> en <code>scripts/win/common.ps1</code>	Fija la raíz de instalación para el script
CHOFYAI_MODELS_DIR	<code>apply_path_env</code>	<code>resolve_models_dir</code> (Bash) / <code>ResolveModelsDir</code> (PowerShell)	Override del directorio de modelos
CHOFYAI_OUTPUTS_DIR	<code>apply_path_env</code>	<code>resolve_outputs_dir</code>	Override del directorio de salidas
CHOFYAI_CACHE_DIR	<code>apply_path_env</code>	<code>resolve_cache_dir</code>	Override del directorio de caché
STUDIO_HOME	El usuario, al ejecutar un script a mano	<code>resolve_studio_home</code>	Alternativa a la anterior
CHOFYAI_DISABLE_UV	El usuario	<code>detect_uv</code>	Con valor <code>1</code> , fuerza <code>python -m venv</code> + pip clásico
HF_HOME	<code>scripts/mac/install-qwen3-tts.sh</code>	Cliente de Hugging Face	Ubica la caché de modelos bajo el Studio Home
UV_LINK_MODE	El mismo script, con valor <code>copy</code>	<code>uv</code>	Evita enlaces duros entre sistemas de archivos distintos
GRADIO_SERVER_NAME, GRADIO_SERVER_PORT	El <code>run.command</code> de FaceFusion	Gradio	Fuerzan el bind local y el puerto
CHOFYAI_CHROME	El usuario	<code>findChrome()</code> en <code>scripts/docs/build-pdf.mjs</code>	Ruta al navegador para generar los PDF
CHOFYAI_SKIP_MERMAID	El usuario	<code>ensureMermaid()</code>	Con valor <code>1</code> , genera los PDF sin renderizar diagramas

Regla de precedencia en los scripts, tal como está implementada en `common.sh`: `CHOFYAI_STUDIO_HOME` → `STUDIO_HOME` → el campo `studio_home` leído del `settings.json` → `$HOME/ChofyAISTudio`. Y si el resultado no es un directorio usable, se cae igualmente al valor por defecto, replicando lo que hace Rust.

4. Claves de `localStorage`

Sólo afectan a la interfaz y viven en el perfil del WebView del usuario.

Clave	Valores	Escrita en
ChofyAI Studio · 10 · Configuración <code>chofyai_theme</code>	<code>dark</code> , <code>light</code> , <code>system</code>	v0.5.1 (commit f840055) · 2026-08-28 Efecto de tema en <code>App</code>
<code>chofyai_lang</code>	<code>es</code> , <code>en</code>	<code>setLang</code> en <code>src/i18n.ts</code>
<code>chofyai_onboarding_done</code>	Cualquier valor no vacío	Al terminar el asistente

Todos los accesos están envueltos en `try/catch`: si el almacenamiento no está disponible, la aplicación usa los valores por defecto (`dark`, `es`, `onboarding visible`).

5. Configuración del frontend

5.1 `package.json`

Campo	Valor	Por qué importa
<code>version</code>	<code>0.5.1</code>	Es la versión que se muestra al usuario vía <code>CARGO_PKG_VERSION</code> ... salvo que <code>APP_VERSION</code> en <code>src/App.tsx</code> dice <code>0.5.0</code>
<code>type</code>	<code>module</code>	Todo el proyecto es ESM
<code>packageManager</code>	<code>pnpm@10.29.3</code>	Corepack instala exactamente esa versión; es una medida de cadena de suministro
<code>pnpm.onlyBuiltDependencies</code>	<code>["esbuild"]</code>	Sólo ese paquete puede ejecutar scripts de instalación; bloquea <code>postinstall</code> arbitrarios de dependencias transitivas
<code>scripts</code>	ver 05 · Referencia técnica	Puerta de entrada a todas las tareas

5.2 `.npmrc`

```
registry=https://registry.npmjs.org/
strict-peer-dependencies=false
resolution-mode=highest
audit-level=high
```

`strict-peer-dependencies=false` evita que un desajuste de `peers` detenga la instalación; tiene la contrapartida de esconder incompatibilidades reales. `audit-level=high` alinea la auditoría local con la que ejecuta el workflow de seguridad.

5.3 `tsconfig.json`

Los valores que más consecuencias tienen: `strict: true`, `noEmit: true` (compila sólo para comprobar; el `bundle` lo genera Vite), `moduleResolution: "bundler"`, `jsx: "react-jsx"`, `isolatedModules: true` y

`include: ["src"]` — lo que significa que `scripts/` y los archivos de configuración **no** pasan por el chequeo de tipos.

5.4 vite.config.ts

Puerto `1420` con `strictPort: true` y `host: '127.0.0.1'`. `strictPort` es deliberado: si el puerto está ocupado, Vite falla en vez de cambiarse, porque `devUrl` en `tauri.conf.json` apunta exactamente a ese número. `clearScreen: false` conserva la salida de Cargo en la terminal durante `tauri:dev`.

6. Configuración del backend y del empaquetado

6.1 src-tauri/Cargo.toml

Versión `0.5.1`, edición 2021 y cinco dependencias: `serde`, `serde_json`, `serde_yaml`, `thiserror` y `tauri`, más `walkdir`. `thiserror` está declarado pero no se usa para definir errores propios: todos los comandos devuelven `Result<_, String>`.

6.2 src-tauri/tauri.conf.json

Clave	Valor	Comentario
<code>productName</code> / <code>mainBinaryName</code>	<code>ChofyAI Studio</code>	Nombre del <code>.app</code>
<code>version</code>	<code>0.5.0</code>	No coincide con <code>package.json</code> ni con <code>Cargo.toml</code>
<code>identifier</code>	<code>cl.vladimiracuna.chofyai.studio</code>	Identificador del bundle
<code>build.beforeDevCommand</code> / <code>beforeBuildCommand</code>	<code>pnpm dev:web</code> / <code>pnpm build:web</code>	Acoplan Tauri a pnpm
<code>build.devUrl</code>	<code>http://localhost:1420</code>	Debe coincidir con <code>vite.config.ts</code>
<code>build.frontendDist</code>	<code>../dist</code>	Salida de Vite
<code>app.windows[0]</code>	<code>1400x920, mínimo 1100x760</code>	Ventana principal <code>main</code>
<code>app.security.csp</code>	<code>null</code>	Sin política de contenido; necesario para el <code>iframe</code> , analizado en 11 · Seguridad
<code>bundle.targets</code>	<code>["app", "dmg"]</code>	Artefactos generados
<code>bundle.resources</code>	<code>../apps</code> , <code>../docs</code> , <code>../scripts/mac</code> , <code>../marketplace</code> , <code>../workflows</code> , <code>../storage/state/settings.json</code>	Lo que viaja dentro del <code>.app</code>

El bloque `bundle.resources` es la razón por la que la aplicación empaquetada encuentra los manifiestos y los scripts: cuando `repo_root()` devuelve `None`, todo se resuelve con `BaseDirectory::Resource`. Añadir

una herramienta nueva con un script fuera de `scripts/mac/` haría que funcionara en desarrollo y fallara empaquetada.

ChofyAI Studio · 16 · Configuración

v0.5.1 (commit f840055) · 2026-08-28

6.3 `src-tauri/tauri.macos.conf.json`

Sobrescritura usada por `pnpm tauri:build:mac: minimumSystemVersion: "13.0", bundleVersion: "1"` y la geometría de la ventana del DMG.

6.4 `src-tauri/capabilities/default.json`

Concede `core:default` a la ventana `main`. Es el conjunto mínimo: no hay plugins de Tauri habilitados.

6.5 `Info.plist` y `Entitlements.plist`

`Info.plist` declara soporte de alta resolución, conmutación automática de gráficos y la categoría `public.app-category.developer-tools`.

`Entitlements.plist` habilita tres permisos de firma: `allow-jit`, `allow-unsigned-executable-memory` y `disable-library-validation`. Son necesarios para ejecutar intérpretes y librerías nativas de terceros (Python, MLX, ONNX Runtime), y a la vez relajan protecciones del sistema: es una concesión consciente, documentada aquí para el auditor.

6.6 `.cargo/config.toml`

```
[build]
target-dir = "/tmp/chofyai-target"
```

Existe porque en volúmenes que no son APFS macOS crea archivos AppleDouble (`._*`) que Cargo y Tauri intentan interpretar como TOML o UTF-8 y rompen la compilación. Consecuencia operativa: los artefactos de build **no** están bajo `src-tauri/target/`, sino en `/tmp`, y se pierden al reiniciar el equipo.

7. Configuración de calidad y automatización

Archivo	Contenido relevante
<code>.markdownlint.json</code>	<code>default: true</code> con MD013 (longitud de línea), MD012 (líneas en blanco múltiples) y MD060 desactivadas; MD033 (HTML) permitido; MD024 con <code>siblings_only</code> ; MD041 activa
<code>.markdownlint-cli2.jsonc</code>	Ignora <code>**/._*</code> , <code>node_modules/**</code> y <code>src-tauri/target/**</code>
<code>.markdownlintignore</code>	Mismo propósito, para invocaciones directas del CLI
<code>.gitignore</code>	Excluye <code>node_modules/</code> , <code>dist/</code> , <code>._*</code> , <code>src-tauri/target/</code> , <code>src-tauri/gen/</code> , <code>/tmp/chofyai-target/</code> , <code>*.log</code> , <code>storage/state/settings.local.json</code> y <code>storage/state/runtime/</code>
<code>.github/dependabot.yml</code>	Tres ecosistemas —npm, cargo y github-actions— semanales los lunes, con agrupación de PR para Tauri, React y Vitest

Nota sobre `.gitignore`: `storage/state/settings.json` **sí** está versionado, y el archivo del repositorio contiene rutas del equipo del autor. No es un secreto, pero sí información del entorno; se comenta en [11 · Seguridad](#).

8. Diferencias entre entornos

Aspecto	Desarrollo web (<code>pnpm dev:web</code>)	Escritorio (<code>pnpm tauri:dev</code>)	Producción (<code>.app</code>)
Backend	Ausente	Presente	Presente
<code>inTauri</code>	<code>false</code>	<code>true</code>	<code>true</code>
Datos mostrados	<code>fallbackTools</code> simuladas	Reales	Reales
<code>repo_root()</code>	—	Devuelve la raíz del repositorio	Devuelve <code>None</code>
Manifiestos	—	<code>apps/</code> del repositorio	Recursos del bundle
<code>settings.json</code>	—	<code>storage/state/</code>	<code><app_data_dir>/state/</code>
Scripts	—	<code>scripts/mac/</code> del repositorio	Recursos del bundle
Directorio de build	—	<code>/tmp/chofyai-target</code>	<code>/tmp/chofyai-target</code>

Esta tabla explica la mayoría de los "en mi máquina funciona" del proyecto: el mismo binario resuelve rutas distintas según cómo se ejecute.

9. Banderas de comportamiento

No hay un sistema formal de *feature flags*. Lo que existe funciona como tal:

Bandera	Dónde	Efecto
CHOFYAI_DISABLE_UV=1	Entorno	Fuerza pip clásico en todas las instalaciones Python
CHOFYAI_SKIP_MERMAID=1	Entorno	Genera los PDF sin renderizar diagramas
chofyai_onboarding_done	localStorage	Oculto el asistente inicial
platforms: en un manifiesto	YAML	Oculto o habilita la herramienta según la plataforma
recommended: en un manifiesto	YAML	Muestra la etiqueta "recomendada"
models: en un manifiesto	YAML	Habilita la descarga guiada de modelos
default_port: en un manifiesto	YAML	Habilita health check por puerto, Ver UI y detección de huérfanos

10. Configuraciones sensibles y consecuencias de un error

Parámetro mal configurado	Síntoma observable	Dónde se manifiesta
<code>studio_home</code> a una ruta inexistente o no escribible	La aplicación arranca en fallback y "desaparecen" las herramientas	<code>resolve_effective_home</code> ; la barra de estado muestra el aviso
<code>studio_home</code> en un volumen exFAT	Fallan los entornos virtuales y los enlaces simbólicos	Scripts de instalación
<code>sparsebundle_path</code> incorrecto	El auto-montaje no ocurre y se cae al fallback sin explicación	<code>resolve_effective_home</code>
<code>tool_overrides</code> apuntando a un directorio que ya no existe	La herramienta figura como no instalada	<code>manifest_install_dir</code> + <code>installed_if</code>
<code>models_dir</code> en un disco lento o desconectado	Descargas y arranques lentos o fallidos	Variables <code>CHOFYAI_*_DIR</code> en los scripts
<code>default_port</code> duplicado entre dos manifiestos	Una herramienta mata a la otra al arrancar	Pre-flight de puerto en <code>start_tool</code>
<code>installed_if</code> mal declarado	La herramienta nunca se da por instalada, o se da por instalada estando rota	<code>list_tools</code> , <code>run_install_script</code> , <code>start_tool</code>
<code>run.command</code> con rutas equivocadas	El proceso arranca y muere; el log queda casi vacío	<code>start_tool</code> , <code><tool>-run.log</code>
<code>devUrl</code> y el puerto de Vite desalineados	Ventana en blanco en <code>tauri:dev</code>	Tauri
<code>bundle.resources</code> sin un directorio nuevo	Funciona en desarrollo y falla empaquetado	<code>script_path</code> , <code>app_paths</code>
Categoría o runtime fuera de la lista permitida	Falla el job <code>validate-manifests</code> en CI	<code>.github/workflows/ci.yml</code>

11. Gestión de secretos

El repositorio no debe contener secretos, y no se han identificado credenciales en el código de la aplicación. Los únicos secretos del proyecto son los de CI/CD para firma y notaría de macOS, referenciados como `secrets.*` en `.github/workflows/release.yml` y descritos por nombre en `../NOTARIZATION.md` . Nunca deben escribirse en `settings.json` , en un manifiesto ni en un workflow de `workflows/` , porque esos archivos se versionan y además viajan dentro del `.app` .